Project 1: User Management App

API: https://dummyjson.com/users

Objective

Build a **User Management System** using:

- HTML
- Tailwind CSS
- Vanilla JavaScript
- Fetch API
- Full CRUD (Create, Read, Update, Delete)

API Documentation:
DummyJSON

Base URL:

https://dummyjson.com

Project Structure

```
user-management/
|
 ├── index.html
 ├── js/
|   └── app.js
└── css/ (if needed)
```

Features To Implement

Display All Users (READ)

On page load:

GET https://dummyjson.com/users

Display users in:

- Table format
- Show:
    - ID
    - First Name
    - Last Name
    - Email
    - Phone
    - Actions (Edit | Delete)

Add New User (CREATE)

Create a form with:

- First Name
- Last Name
- Email
- Phone

On submit:

POST https://dummyjson.com/users/add

Send body as JSON:

```
{
  "firstName": "Ali",
  "lastName": "Khan",
  "email": "ali@example.com",
  "phone": "123456789"
}
```

After success:

- Show success message
- Add user to table (without refreshing page)

- Reset form

Edit User (UPDATE)

When clicking Edit:

- Open modal OR fill form with selected user data

Send:

PUT https://dummyjson.com/users/{id}

Example:

PUT https://dummyjson.com/users/1

After success:

- Update user in UI
- Close modal
- Show success alert

Delete User (DELETE)

When clicking Delete:

DELETE https://dummyjson.com/users/{id}

After success:

- Remove user row from table
- Show confirmation message

Search Users (BONUS)

Use:

GET https://dummyjson.com/users/search?q=John

Create:

- Search input

- Filter results dynamically

UI Requirements

- Use Tailwind CSS
- Responsive layout
- Styled table
- Styled form
- Modal for editing (optional but recommended)
- Loading spinner while fetching data

Important Notes

- DummyJSON does NOT permanently save data
- You must manually update the UI
- Use:
    - async/await
    - try/catch
    - Proper error handling

Concepts You Must Use

- fetch()
- JSON
- DOM manipulation
- Event listeners
- Prevent default form submission
- Dynamic rendering
- Conditional rendering

Project 2: Product Inventory App

API: https://fakestoreapi.com/products

API Provider:
Fake Store API

Objective

Build a **Product Inventory Management System**

Project Structure

```
product-inventory/
│
├── index.html
├── app.js
```

Features To Implement

Display All Products (READ)

On page load:

GET https://fakestoreapi.com/products

Display in:

- Card layout OR Table
- Show:
    - Image
    - Title
    - Price

    - Category
    - Description (shortened)
    - Actions (Edit | Delete)

Add Product (CREATE)

Form Fields:

- Title
- Price
- Description
- Category
- Image URL

Send:

POST https://fakestoreapi.com/products

Example body:

```
{
  "title": "New Product",
  "price": 29.99,
  "description": "Product description",
  "image": "https://i.pravatar.cc",
  "category": "electronics"
}
```

After success:

- Add new product to UI
- Show success message
- Reset form

Update Product (UPDATE)

PUT https://fakestoreapi.com/products/{id}

After success:

- Update UI
- Show confirmation

Delete Product (DELETE)

DELETE https://fakestoreapi.com/products/{id}

After success:

- Remove product from UI

Filter by Category (BONUS)

GET https://fakestoreapi.com/products/category/{category}

Add dropdown to filter:

- electronics
- jewelery
- men's clothing
- women's clothing

UI Requirements

- Tailwind CSS
- Responsive grid
- Hover effects on cards
- Low price highlight (< $20)
- Loading spinner
- Empty state message

Mandatory Technical Requirements (Both Projects)

Students MUST:

- ✅ Use async/await
- ✅ Use try/catch
- ✅ Show loading indicator
- ✅ Show error message on failure
- ✅ Update UI without refreshing
- ✅ Separate JS file
- ✅ Clean, readable code